

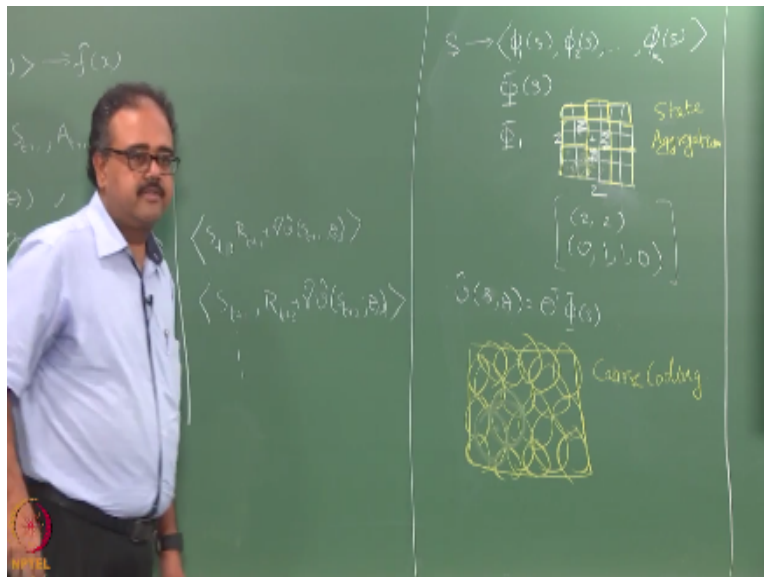
NPTEL
NPTEL ONLINE COURSE
REINFORCEMENT LEARNING

State Aggregation Methods

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

Let us see how far we go okay so one of the most, okay, lookup tables right, I tell you that a lookup table is a linear function approximator, do you agree with me or not right how what will be my ϕ , ϕ will be indicator function right, so ϕ_i of s will be 1 if s equal to s_i right so that essentially becomes a linear function approximator, right so well you do not want to use that is a useless function approximator but just useful to think about it that way now I can define another form of indicator function approximator right known as state aggregation.

(Refer Slide Time: 01:16)



So what is state aggregation let us say that I have a very very large state space right, a very very large state space, I can typically assume that states that are close by will have the same value

right suppose I have million by million grid world right if I assume that states that are say within 10 of each other right like a 10 by 10 grid, I can take any 10 by 10 grid and the states will have the same value within the 10 by 10 grid right, why, that is, you are moving into nearest neighbor kind of a setup, we are not talking about that, I am just saying that given that I have to reduce this, I have a million by million right.

So how much is my value function going to change if I move 10 states in one direction I have like another nine hundred and ninety nine thousand nine hundred ninety states there to go, I am trying to reduce the dimensionality I am trying to reduce K right, so if I had done a lookup table right my K will be a million square right I am trying to reduce the dimensionality of that right I am going to say that way, it is another kind of indicator function instead of saying that it is exactly this state I am going to say it is one of these 100 states, it is still on policy, I am, the samples are still on policy right the sample generation is still on policy, no no no, I am just sampling a trajectory right following π , it is not like I am jumping around and sampling things there is some amount of reduction of information here right but still the sampling is still done on policy right.

So I can use this indicator function which says I am in this this big grid right so I can take this large okay let me take this large grid world right it has to have odd number of, now I break it up into instead of having 30 indicator functions I am going to have only 8, 9 indicator functions right instead of 30 indicator functions I will have 9 indicator functions right so this kind of an approximation is called state aggregation, this kind of approximation is called state aggregation. Because it is talking about grid worlds I am drawing squares right.

If I am talking about say a continuous world right so for example talking about like robot navigation problem or an arm manipulator, things like that where my states is given by joint angles or something like that right I am going to have a continuous state space right so I can talk about any kind of compact state right on in that state space and I can talk about memberships of that right so for example I can take this is a continuous state space right I can start talking about right I am going to put this kind of circles on the on the state space right.

So all states here this is an indicator function basically for all the states here this will be one, for all the states here this will be one, for all the states here this will be one so what is the problem with this it is not covering the whole space right so typically what happens is we end up actually covering the whole space, okay, now what is the problem with that, overlaps, so what is the big deal with overlaps, two indicator functions are going to come instead of one, it is better, why, the generalization would be better.

See earlier I was telling you when we did the grids right when we did the state aggregation I was telling you that any block of ten states I will take, it will have the same value right but that is not true in the representation I am using here because there is going to be discontinuity here right these four states will get one value because the average of these 4 that is essentially what will happen right it will learn to learn to be the average value if you look at how the update happens right.

So every time I come I will update the same value right so it will learn to be the average of this 4 and this will learn to be the average of this 4 but these 2 which are actually right next to each other we will have a drastically different value right when I start doing this overlap so what will happen is well these will get updates together likewise these will get updated together but the states here will actually get updated for both essentially it will convert to the average of both right.

So it will not be a abrupt shift from one to the other, it will be more like a well step okay, it will not be a graded shift, there will be a base like this, there will be one intermediate value which is the average of both and then another one so it will not be a complete jump it will be a gradual jump right so this is okay to have this kind of overlaps in fact it could you could argue that it is desirable to have these kinds of overlaps right and this way of and again we did nothing here, nothing fancy here all of these are indicator functions still, except that depending on the degree of overlap that many indicator functions will come on for a state right it could be one it could be two it could be three it could be more right.

Why, compute the what, domain knowledge intuition, tarot cards, I mean this is why deep learning is becoming popular nowadays because you can use deep learning instead of the tarot cards right so you have to use some kind of in this case I am assuming all of this are geographic, I mean geometric sorry geometric problems right so I am assuming nearness in the particular metric space I am looking at indicates nearness in the value function it will not always be the case right so there are people who actually look at so the way to address this is to take the state space that you have embed it in some manifold.

Where the nearness in the distance actually means that you are near in value function as well right so there are many methods which people have used, that they have tried to come up with different kinds of basis functions which have that kind of , the basis function would be what, what are my basis functions here this particular case, whatever we are talking about what are my basis functions, yeah the ϕ_1, ϕ_2, ϕ_K , so I am using some K basis functions here so people have come up with different ways of transforming this basis.

So that nearness in that transformed space right indicates nearness in the value function as well right so this is one example that I was giving yesterday right so take a grid world or take some space like this right let us say I have a some goal there right and say my parameterization consists of the X and the Y and for every time step I have not reached the goal I get a reward of -1 right so how will my value function look like, well in the negative direction assuming the negative direction comes out of the board right so it is going to look like this right so it is going to slide all the way in so the peak will be somewhere here right and then from here there will be a gradual decrease all the way to this point correct.

Right so that is a nice smooth value function right in fact I can think of depending on my dynamics and stuff like that, that could even be linear right, so it could be just a plane tipping so it could be a plane that is tipped over like this right that should be fine, I take a sheet like this and then I can probably represent it using a linear parameterization great now what I am going to do is, I am going to put a wall now what is going to happen to this there will be some kind of a discontinuity, so where will the discontinuity be so here it will be nice and small thing here it will be a much larger value right.

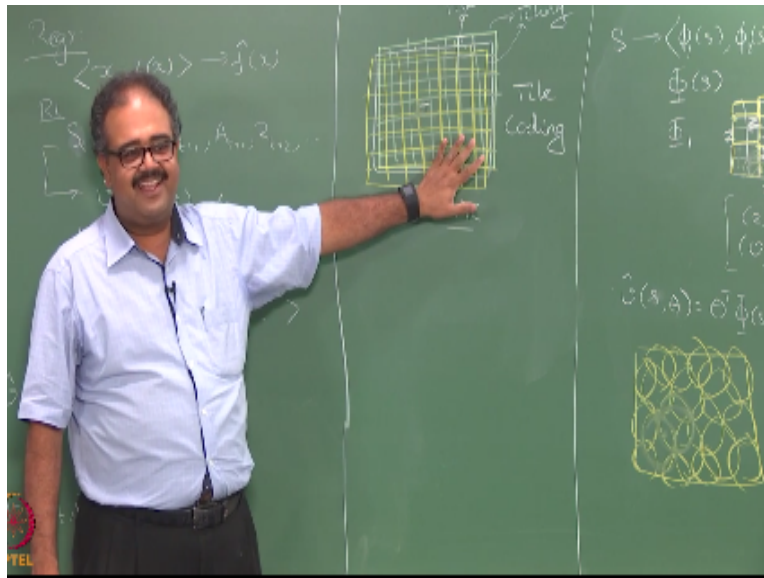
So one way to fix this is to think of some kind of an embedding where I essentially split the room open like that right so I essentially think of, in reality that is what it is right, if you think about it and now it is in fact I made it into some kind of a quasi corridor by drawing the line there right so I can think of transforming the space into something like this, now if I look at the XY coordinates here right it might be a good enough approximation for the value function right so it could be a good enough nearness estimate, I mean obviously this is not what you have to do you have to look at a more complex embedding, I am just trying to give you some intuition as to what you try to do when you find those embeddings okay.

You try to get a transformed space so that in fact if you think about it so my y-coordinate runs like that here so it is not a proper linear thing so this will be my y coordinate right so it is something like this I will have to think about right, so it is a complex thing so what people do work on stuff like this right they look at all kinds of different basis so that you can do this kind of generalization at the end of the day so we will not worry about that, let us stay in the, let us stay in a more innocent world for the time being where we assume that this kind of a generalization is fine without having to worry about any other basis transformations okay.

So this is called coarse coding, so I am doing some very coarse representation of the state right sometimes called coarse coding so the problem with coarse coding is that there is no uniformity in the number of bits that I am using for, I mean number of on bits right I am using for representing a state so some states could have only one bit some states could have multiple bits and so on so forth and so you could people came up with a more systematic way of doing this coarse coding okay.

And as many of the things as described in the book this is not the way they came up with whatever I am going to talk about right so they do something called tile coding so I am just going to use this space because it is the last thing I am going to do today so I will stop with this.

(Refer Slide Time: 14:32)



So let us consider a some input space right so tile coding does the following right is splits it up into the kind of regular tiles okay but this is not giving you that nice overlap features that the coarse coding had, it is just almost like state aggregation, so what tile coding additionally does is that it does not do one such grid it does multiple such grids, okay, it has got really confusing while it is very very simple, if you think about any state any point in this state space it will always light up two of the indicator functions one from the green grid one from the yellow grid right.

And it gives you this overlap that you wanted right so all the states in this yellow grid will get updates from this place and all the states in this green grid will get this update from this state so in some sense you get the overlap right so there is some set of states here which contribute to this which do not contribute under the green grid, so there is some amount of overlap that happens from one cell to the other from the green to the yellow and vice versa it gives you the advantage of coarse coding.

It so it is a form of coarse coding but it is more systematic because every state will have two indicator functions lighting up right so each one of these is called a tiling right so that is a tiling and that is a tiling and each one of this is a, any guesses, so each one of those is a tile and the whole grid that we talked about is called the tiling so in this example I used two tilings people typically use like anywhere between four to seven tilings to get good performance right and so how many indicator functions light up for a given input state, number of tilings, right so as many tilings that there are, if I have seven tilings there will be 7 things that light up right.

So it turns out to be a very very good function approximator for many RL tasks that people were looking at about 2, 3 years ago till about 2, 3 year ago, so this turned out to be a really good so obviously you cannot play go or backgammon using tile coding right you need something more cleverer than that, right it turns out that you need something more so that is the biggest advantage of all this deep learning methods is that you do not have to define the ϕ , okay they find the ϕ from the data that is the biggest advantage of many of these tile coding methods, they just find

What are the appropriate ϕ from the data itself and then uses that ϕ to define a value function and then learn those value functions, everything is done in one shot it is not like I first learn the ϕ and then learn the value function or anything, it is a single network as I keep training it, it learns the ϕ and the value function also, it is an amazing thing right, looks like we are getting a lot of advantages for free, what is the downside you need significantly powerful computing equipment first otherwise it is not going to run anytime soon and you need a lot of data.

So that is the, some of the biggest challenges so tile coding, one popular instance of tile coding was a specific thing called CMAC so CMAC stands for you should tell me now, Cerebellar Model Actuation Controller right so essentially it was originally proposed as a model of the cerebellum, I mean a human cerebellum right and so from there, cerebrum or cerebellum and then from there I mean of course it is I do not know if people still consider.

It as a serious model of the cerebellum but this comes from way before right, 60s, 50s and 60s okay this is the guy called Albus who proposed the CMAC and it turns out that CMACs do some kind of tile coding and then they do a whole bunch of tricks to compress the tilings right so

nowadays with a lot of memory at disposal you do not would not want to compress the tilings but the classical CMAC implementation uses a lot of hashing based tricks to compress the tile tiling representation.

So that you do not have to store the entire vector theta, right if you think about it suppose each tiling is 10 by 10 right so I have number of tilings into hundred thetas right and if the tilings become more and more, more finer right and state space is very large and I want to keep smaller tiles right then the number of thetas that I need to store becomes larger and larger right I do not actually have to store the tiles themselves because they are so regular.

I can have some kind of a function right that computes the tile memberships I can have some kind of I can have a functional form that will tell me which tiles come on right but I have to store the thetas, right at the end of the day this is also a lookup table right except that I determine the membership which entry I look up in the table in a very complex systematic but complex way right instead of just looking at the index, right so people use hashing and you know you know you can you can use hashing to compress tables.

So just like that they use hashing to compress the entries in this and so it has advantages and disadvantages in fact it gives you unintended generalization right because two different tiles could be hashed to the same weight right so one tile from here right, one tile from here another tile from here could hash to the same weight that gives you some kind of destructive generalization right but then there are some conditions under which you can show that this is not too bad and so on so forth.

But I personally do not like hashing but we have found out that in some cases in very large state spaces, having hashing actually helps you to learn faster so really have not managed to explain why that is the case here right so I will stop here then so I will talk about eligibility traces and control and other things in the next class.

Funded by
Department of Higher Education
Ministry of Human Resource Development
Government of India

www.nptel.ac.in

Copyrights Reserved